

# FRAQ: Efficient Coordinate-Space Recompression for Federated Low-Rank Adaptation

Shenghui Li<sup>1</sup> and Thiemo Voigt<sup>1,2</sup>

<sup>1</sup>Uppsala University, Uppsala, Sweden

<sup>2</sup>Research Institutes of Sweden, Stockholm, Sweden  
{shenghui.li, thiemo.voigt}@angstrom.uu.se

## Abstract

Federated fine-tuning with Low-Rank Adaptation (LoRA) enables collaborative and efficient adaptation of Large Language Models (LLMs) without centralizing private data. However, its two-factor parameterization introduces an aggregation mismatch across clients, as naive factor-wise averaging does not recover the average of the induced updates. Forming the exact aggregate in the full weight space and recompressing it avoids this mismatch, but decomposing the resulting full-size matrix is computationally expensive and memory-intensive. We propose FRAQ, an efficient coordinate-space recompression method for federated LoRA. Starting from stacked factors that represent the exact aggregate, FRAQ expresses the aggregate as the product of an orthonormal basis and a compact coordinate matrix. It then recovers the singular spectrum from a small Gram matrix of this coordinate representation and selects the smallest rank satisfying a prescribed energy threshold. The selected coordinate subspace is mapped back via the basis to construct the global adapter. Experiments across text classification and commonsense reasoning benchmarks demonstrate that FRAQ achieves accuracy close to that of uncompressed baselines while substantially reducing downlink communication and incurring low server-side recompression overhead.

## 1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across a wide range of tasks (Brown et al., 2020; Touvron et al., 2023; Grattafiori et al., 2024; Qwen Team, 2025). While fine-tuning further improves the adaptability of LLMs to downstream domains (Howard and Ruder, 2018), fully updating their massive parameter sets remains computationally expensive and susceptible to overfitting. Parameter-Efficient Fine-Tuning (PEFT) methods alleviate these limitations

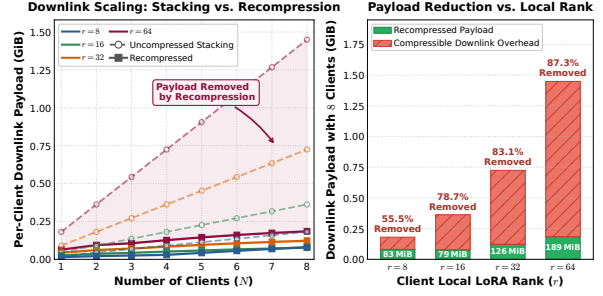


Figure 1: Exact aggregation via unrecompressed stacking rapidly increases per-client downlink with client count and local rank. On LLaMA-3.2-3B, recompression with at least 90% spectral-energy retention cuts this payload by 55.5–87.3%.

by adapting LLMs with only a small number of trainable parameters, with Low-Rank Adaptation (LoRA) (Hu et al., 2022) being a representative approach that introduces trainable low-rank updates to existing weight matrices.

When the data required for fine-tuning LLMs are distributed across institutions or edge devices, directly centralizing them is often infeasible due to privacy or regulatory constraints. Federated Learning (FL) (McMahan et al., 2017) provides a natural paradigm for collaborative learning without sharing local data. When combined with LoRA, FL allows clients to train and communicate only compact low-rank adapters rather than full model parameters, substantially reducing communication overhead. However, LoRA also makes aggregation more subtle: conventional FedAvg-style aggregation (Zhang et al., 2024) averages the two LoRA factor matrices independently, although the product of averaged factors generally differs from the average of the corresponding low-rank updates. This mismatch produces an inexact global update and can degrade fine-tuning performance (Sun et al., 2024; Singhal et al., 2025).

To address this mismatch, existing methods either constrain the local LoRA parameteriza-

tion (Sun et al., 2024) or move to update-level aggregation through residual merging (Singhal et al., 2025) or factor stacking (Wang et al., 2024). The former limits local adaptation, whereas update-level methods better preserve the aggregate without necessarily producing a compact rank-adaptive global adapter. In particular, unrecompressed stacking exactly represents the aggregate, yet its adapter width grows with the sum of participating client ranks (Wang et al., 2024; Singhal et al., 2025), increasing the per-client downlink payload and local computation cost. Intuitively, the stacked width can be much larger than the number of directions needed to capture most of the aggregate’s spectral energy (Yan et al., 2026; Ramesh and Dass, 2026a). The server can therefore recompress the aggregate into a lower-rank adapter before broadcasting it. As illustrated in Figure 1, recompression removes much of the downlink overhead while retaining most of the spectral energy.

However, conventional approaches materialize the dense aggregate in the full weight space and apply truncated SVD (Bai et al., 2024; Singhal et al., 2025), incurring substantial server-side memory and computation. Recent compact-space variants avoid dense reconstruction, yet still require repeated truncated SVDs (Panariello et al., 2025; Ramesh and Dass, 2026a) or two reduced QR factorizations followed by an SVD of the compact core (Ramesh and Dass, 2026b).

We propose FRAQ, a direction-adaptive, one-sided coordinate-space recompression framework for federated LoRA. FRAQ confines orthogonalization to the smaller weight dimension, yielding an implicit basis and exact coordinates. It then recovers the spectrum through a compact Gram eigenproblem, selects the global rank using an energy threshold, and maps only the retained directions back through the implicit basis to form an optimal rank- $p$  adapter. Our main contributions are summarized as follows:

- We establish an isometric coordinate formulation for federated LoRA recompression, proving that compact coordinates of the exact stacked aggregate preserve its nonzero singular spectrum and low-rank approximation geometry. This guarantees full-space rank- $p$  optimality after coordinate-space truncation.
- We design FRAQ, a direction-adaptive, one-sided algorithm that orients the factorization

along the smaller weight dimension, constructs an implicit basis with a single reduced QR, and performs rank-adaptive recompression in compact coordinate space. FRAQ thereby avoids materializing and decomposing the full-size aggregate while remaining equivalent to full-space recompression.

- Extensive experiments across multiple tasks under varying data heterogeneity and both uniform and heterogeneous client ranks show that FRAQ maintains accuracy close to that of uncompressed aggregation while substantially reducing downlink communication and incurring low server-side recompression overhead.

## 2 Preliminaries

**Fine-tuning with LoRA.** LoRA (Hu et al., 2022) represents the update of a pretrained weight matrix through a low-rank factorization. The fine-tuned weights  $\mathbf{W}'$  are expressed as the sum of the frozen pretrained weights  $\mathbf{W}_0$  and a low-rank update  $\Delta\mathbf{W}$ :

$$\mathbf{W}' = \mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + s\mathbf{B}\mathbf{A}, \quad (1)$$

where  $\mathbf{W}_0, \mathbf{W}' \in \mathbb{R}^{m \times n}$  are the pretrained and fine-tuned weights,  $\mathbf{B} \in \mathbb{R}^{m \times \rho}$  and  $\mathbf{A} \in \mathbb{R}^{\rho \times n}$  are the trainable low-rank factors, and  $s = \alpha_{\text{LoRA}}/\rho$  is the LoRA scaling factor. Thus, the effective weight update is  $\Delta\mathbf{W} = s\mathbf{B}\mathbf{A}$ . Instead of updating  $\mathbf{W}_0$  directly, LoRA trains only the small factors  $\mathbf{B}$  and  $\mathbf{A}$ , reducing the number of trainable parameters from  $mn$  to  $\rho(m + n)$ .

**Federated fine-tuning.** Federated learning (McMahan et al., 2017) trains a shared model across a set  $\mathcal{C}$  of  $K$  clients holding private data, coordinated by a central server over  $T$  communication rounds. Under FedAvg, the server aggregates local updates by a data-weighted average; with  $n_k$  samples at client  $k$ ,  $N = \sum_k n_k$ , and weight  $\alpha_k = n_k/N$ , the global update is

$$\overline{\Delta\mathbf{W}} = \sum_{k=1}^K \alpha_k \Delta\mathbf{W}_k, \quad (2)$$

where  $\Delta\mathbf{W}_k$  is the local update from client  $k$ . FedIT (Zhang et al., 2024) extends this scheme to LoRA: each client  $k$  fine-tunes local adapters  $(\mathbf{B}_k, \mathbf{A}_k)$ . FedIT assumes homogeneous LoRA configurations, so all clients use a common LoRA

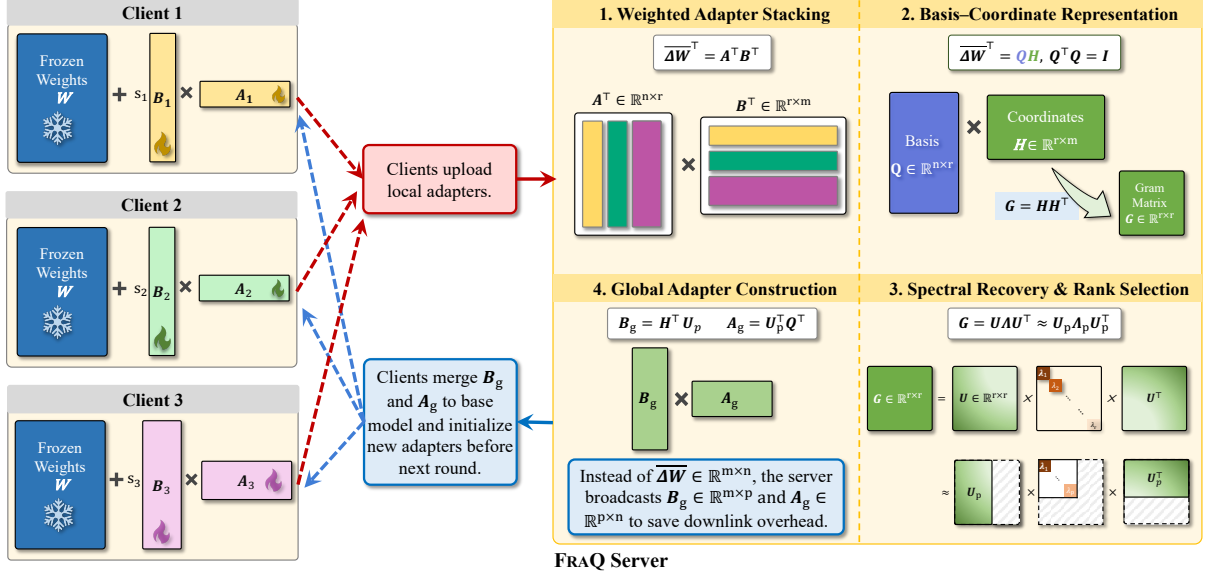


Figure 2: Overview of FRAQ, illustrated for the  $m > n$  orientation. Each client initializes and trains a LoRA adapter on the current global model and uploads its factors to the server. The server (1) forms weighted factor stacks whose product equals the exact update-level aggregate; (2) constructs an orthonormal basis and a coordinate representation without materializing the full-size aggregate; (3) recovers the spectrum from a small Gram matrix and selects the global rank using an energy-retention threshold; and (4) combines the retained coordinate-space approximation with the basis to construct compact global LoRA factors for broadcast. Clients merge the broadcast global adapter into their cached global weights and reinitialize fresh local adapters for the next round.

scale  $s$ . The server averages the factors independently,

$$\bar{B} = \sum_{k=1}^K \alpha_k B_k, \quad \bar{A} = \sum_{k=1}^K \alpha_k A_k. \quad (3)$$

**Inexactness of LoRA aggregation.** The difficulty is that aggregating adapters is not equivalent to aggregating their updates. Because each client’s effective contribution is  $sB_k A_k$  rather than the individual factors, independently averaging  $B_k$  and  $A_k$  does not recover the exact aggregated update:

$$\underbrace{s \overline{B} \overline{A}}_{\text{FedIT: inexact}} = s \sum_{i,j} \alpha_i \alpha_j B_i A_j \neq s \underbrace{\sum_k \alpha_k B_k A_k}_{\overline{\Delta W}: \text{exact}}. \quad (4)$$

The discrepancy stems from the cross terms  $B_i A_j$  with  $i \neq j$ , i.e., the average of the products is not equal to the product of the averages. A naive remedy is to materialize the exact aggregate  $\overline{\Delta W} \in \mathbb{R}^{m \times n}$  in Eq. (4) and re-decompose it with a truncated SVD, but this discards the low-rank structure that makes LoRA efficient and incurs a costly full-matrix decomposition on the server. Our method computes the rank-truncated form of the exact aggregate  $\overline{\Delta W}$  directly in a compact coordi-

nate space, avoiding both the cross-term noise of Eq. (4) and the cost of full-matrix SVD.

### 3 Proposed Method

We introduce FRAQ, an efficient method for consolidating client LoRA updates into a compact global adapter. As illustrated in Figure 2, FRAQ first forms weighted factor stacks whose product equals the exact update-level aggregate and represents this aggregate through an orthonormal basis and a compact coordinate matrix. It then recovers the spectrum from a small Gram matrix formed from the coordinates, uses an energy-retention threshold to select the broadcast rank, and maps the retained directions through the basis to construct the LoRA factors broadcast to clients.

#### 3.1 Problem Formulation

At the beginning of round  $t$ , all clients initialize from the current global weights  $W_t$ , which incorporate all previously merged updates into the pre-trained model  $W_0$ . Client  $k$  then fine-tunes a rank  $r_k$  low-rank adapter  $(B_k, A_k)$ , with  $B_k \in \mathbb{R}^{m \times r_k}$  and  $A_k \in \mathbb{R}^{r_k \times n}$ , on top of  $W_t$ , yielding the effective local update  $\Delta W_k = s_k B_k A_k$ , where  $s_k = \alpha_{\text{LoRA},k}/r_k$ . For clarity, we omit round indices on adapter factors throughout Sections 3.1–

3.3; Figure 2 follows the same convention. With aggregation weights  $\alpha_k = n_k/N$ , the server seeks the weighted aggregate of these updates,

$$\overline{\Delta \mathbf{W}} = \sum_{k=1}^K \alpha_k \Delta \mathbf{W}_k = \sum_{k=1}^K \alpha_k s_k \mathbf{B}_k \mathbf{A}_k. \quad (5)$$

Averaging  $\mathbf{B}_k$  and  $\mathbf{A}_k$  separately introduces the cross-term noise of Eq. (4) (Sun et al., 2024). Instead, following the exact weighted stacking scheme of FLoRA (Wang et al., 2024) and FLoRIST (Ramesh and Dass, 2026a), the server constructs

$$\begin{aligned} \mathbf{B} &= [\sqrt{\alpha_1} \mathbf{B}_1, \dots, \sqrt{\alpha_K} \mathbf{B}_K] \in \mathbb{R}^{m \times r}, \\ \mathbf{A} &= [\sqrt{\alpha_1} s_1 \mathbf{A}_1^\top, \dots, \sqrt{\alpha_K} s_K \mathbf{A}_K^\top]^\top \in \mathbb{R}^{r \times n}. \end{aligned} \quad (6)$$

where  $r = \sum_{k=1}^K r_k$  is the total client rank. The client-specific LoRA scale is incorporated into the stacked  $\mathbf{A}$  factor, matching the implementation. Consequently,  $\mathbf{B}\mathbf{A} = \sum_k \alpha_k s_k \mathbf{B}_k \mathbf{A}_k = \overline{\Delta \mathbf{W}}$ . The aggregate is therefore represented exactly as

$$\overline{\Delta \mathbf{W}} = \mathbf{B}\mathbf{A}, \quad \text{rank}(\overline{\Delta \mathbf{W}}) \leq r, \quad (7)$$

with heterogeneous client ranks  $r_k$  accommodated naturally by the concatenation. This exactness, however, comes at a cost: stacking inflates the individual rank- $r_k$  adapters into an aggregate of rank up to the total client rank  $r$ , which grows with the number of participating clients. To keep the downlink compact, the server must instead broadcast a rank- $p$  adapter  $\mathbf{B}_g \in \mathbb{R}^{m \times p}$  and  $\mathbf{A}_g \in \mathbb{R}^{p \times n}$ , where  $p \ll r$ . The problem to solve is therefore the *recompression* of the exact aggregate: find the rank- $p$  adapter closest to  $\overline{\Delta \mathbf{W}}$  in Frobenius norm,  $\min_{\mathbf{B}_g, \mathbf{A}_g} \|\overline{\Delta \mathbf{W}} - \mathbf{B}_g \mathbf{A}_g\|_F$ , or equivalently

$$\min_{\text{rank}(\mathbf{Z}) \leq p} \|\overline{\Delta \mathbf{W}} - \mathbf{Z}\|_F. \quad (8)$$

### 3.2 Isometric Coordinate Representation

For any compatible matrix product  $\mathbf{M} = \mathbf{X}\mathbf{Y}$ , we have  $\text{col}(\mathbf{M}) \subseteq \text{col}(\mathbf{X})$ . If  $\mathbf{X}$  has  $r$  columns, then  $\text{rank}(\mathbf{M}) \leq \text{rank}(\mathbf{X}) \leq r$ . Thus, although  $\mathbf{M}$  may be full-sized, its columns lie in a subspace of dimension at most  $r$ . Representing  $\mathbf{M}$  in an orthonormal basis whose span contains this subspace is therefore exact. The following proposition shows that the resulting basis-coordinate mapping is isometric, allowing optimal low-rank recompression to be performed equivalently in the coordinate and original matrix spaces.

#### Proposition 1 (Isometric coordinate equivalence).

Let  $\mathbf{M} = \mathbf{X}\mathbf{Y}$ , where  $\mathbf{X} \in \mathbb{R}^{d \times r}$  and  $\mathbf{Y} \in \mathbb{R}^{r \times D}$ , with  $r \leq d$ . Let  $\mathbf{Q} \in \mathbb{R}^{d \times r}$  have orthonormal columns whose span contains  $\text{col}(\mathbf{X})$ , i.e.,  $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_r$  and  $\text{col}(\mathbf{X}) \subseteq \text{col}(\mathbf{Q})$ . Define

$$\begin{aligned} \mathbf{C} &= \mathbf{Q}^\top \mathbf{X} \in \mathbb{R}^{r \times r}, \\ \mathbf{H} &= \mathbf{C}\mathbf{Y} = \mathbf{Q}^\top \mathbf{M} \in \mathbb{R}^{r \times D}. \end{aligned} \quad (9)$$

Then  $\mathbf{M} = \mathbf{Q}\mathbf{H}$ . Moreover, for any  $\widehat{\mathbf{H}} \in \mathbb{R}^{r \times D}$ ,

$$\|\mathbf{M} - \mathbf{Q}\widehat{\mathbf{H}}\|_F = \|\mathbf{H} - \widehat{\mathbf{H}}\|_F. \quad (10)$$

Thus, the columns of  $\mathbf{H}$  are the exact coordinates of the corresponding columns of  $\mathbf{M}$  in the basis  $\mathbf{Q}$ , and  $\mathbf{H}$  and  $\mathbf{M}$  have the same nonzero singular values. Consequently, if  $\mathbf{H}_p$  is a best rank- $p$  approximation of  $\mathbf{H}$ , then

$$\mathbf{Q}\mathbf{H}_p \in \arg \min_{\substack{\widetilde{\mathbf{M}} \in \mathbb{R}^{d \times D} \\ \text{rank}(\widetilde{\mathbf{M}}) \leq p}} \|\mathbf{M} - \widetilde{\mathbf{M}}\|_F. \quad (11)$$

Proposition 1 shows that optimal rank- $p$  recompression can be performed on  $\mathbf{H}$  in the  $r$ -dimensional coordinate space induced by  $\mathbf{Q}$ , without materializing the full matrix  $\mathbf{M}$ . The proposition is stated under the assumption  $r \leq d$ , which holds in all federated LLM fine-tuning settings evaluated in this work. The extension to  $r > d$  is provided in Appendix A.2, and the proof of the proposition is provided in Appendix A.1.

### 3.3 FRAQ Recompression

Given the weighted factor stacks in Eq. (6), FRAQ instantiates Proposition 1 to recompress the exact aggregate. Because transposition preserves singular values, rank, and the Frobenius norm, FRAQ can operate on either the aggregate or its transpose. We therefore orient the factorization along the smaller weight dimension:

$$(\mathbf{X}, \mathbf{Y}) = \begin{cases} (\mathbf{B}, \mathbf{A}), & m \leq n, \\ (\mathbf{A}^\top, \mathbf{B}^\top), & m > n. \end{cases} \quad (12)$$

Accordingly,  $\mathbf{M} = \mathbf{X}\mathbf{Y}$  equals  $\overline{\Delta \mathbf{W}}$  when  $m \leq n$  and  $\overline{\Delta \mathbf{W}}^\top$  when  $m > n$ . In either case,  $\mathbf{X} \in \mathbb{R}^{d \times r}$  and  $\mathbf{Y} \in \mathbb{R}^{r \times D}$ , where  $d = \min(m, n)$  and  $D = \max(m, n)$ .

**Basis-coordinate representation.** To obtain the orthonormal basis required by Proposition 1, FRAQ factorizes the tall-and-skinny left factor  $\mathbf{X} \in \mathbb{R}^{d \times r}$ .

We use a reduced Householder QR factorization as the default exact basis construction, obtaining  $\mathbf{R} \in \mathbb{R}^{r \times r}$  and an implicit representation of  $\mathbf{Q} \in \mathbb{R}^{d \times r}$  such that  $\mathbf{X} = \mathbf{Q}\mathbf{R}$  and  $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_r$ . Because  $\text{col}(\mathbf{M}) \subseteq \text{col}(\mathbf{X}) \subseteq \text{col}(\mathbf{Q})$ , the columns of  $\mathbf{M}$  are represented exactly in the basis  $\mathbf{Q}$ . Substituting  $\mathbf{X} = \mathbf{Q}\mathbf{R}$  into  $\mathbf{M} = \mathbf{X}\mathbf{Y}$  and defining  $\mathbf{H} = \mathbf{R}\mathbf{Y}$  gives  $\mathbf{M} = \mathbf{Q}\mathbf{H}$ . Whereas existing approaches to LoRA merging and recompression, such as Core Space Merging (Panariello et al., 2025) and FLoRIST (Ramesh and Dass, 2026a), construct left and right bases using a pair of thin SVDs, FRAQ obtains the required basis from a single reduced Householder QR factorization of  $\mathbf{X}$  and incorporates  $\mathbf{Y}$  directly through  $\mathbf{R}\mathbf{Y}$ .

**Spectral recovery and rank selection.** Since Proposition 1 establishes that  $\mathbf{H}$  and  $\mathbf{M}$  share the same nonzero singular values, FRAQ performs spectral analysis entirely in the coordinate space. Rather than computing an SVD of  $\mathbf{H} \in \mathbb{R}^{r \times D}$ , it forms the compact left Gram matrix and computes its symmetric eigendecomposition:

$$\mathbf{G} = \mathbf{H}\mathbf{H}^\top = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top \in \mathbb{R}^{r \times r}. \quad (13)$$

Here  $\mathbf{U}^\top \mathbf{U} = \mathbf{I}_r$  and  $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_r)$ , with  $\lambda_1 \geq \dots \geq \lambda_r \geq 0$ . Because  $\mathbf{G} = \mathbf{H}\mathbf{H}^\top$ , the columns of  $\mathbf{U}$  are the left singular vectors of  $\mathbf{H}$ , and  $\lambda_i = \sigma_i(\mathbf{H})^2 = \sigma_i(\mathbf{M})^2$ . Moreover,  $\sum_{i=1}^r \lambda_i = \|\mathbf{H}\|_F^2 = \|\mathbf{M}\|_F^2$ , so each  $\lambda_i$  measures the contribution of the corresponding singular direction to the aggregate’s spectral energy.

For a nonzero aggregate and an energy-retention threshold  $\tau \in (0, 1]$ , FRAQ selects the smallest rank whose leading directions retain at least a  $\tau$  fraction of this energy:

$$p = \min \left\{ p' \in \{1, \dots, r\} \left| \frac{\sum_{i=1}^{p'} \lambda_i}{\sum_{i=1}^r \lambda_i} \geq \tau \right. \right\}. \quad (14)$$

**Global adapter construction.** Let  $\mathbf{U}_p = [\mathbf{u}_1, \dots, \mathbf{u}_p] \in \mathbb{R}^{r \times p}$  contain the eigenvectors associated with the  $p$  largest eigenvalues of  $\mathbf{G}$ . These vectors span the dominant left singular subspace of  $\mathbf{H}$ . Orthogonally projecting  $\mathbf{H}$  onto this subspace therefore gives its best rank- $p$  approximation  $\mathbf{H}_p = \mathbf{U}_p \mathbf{U}_p^\top \mathbf{H}$ . Mapping this approximation back to the original matrix space via  $\mathbf{Q}$  yields

$$\mathbf{Q}\mathbf{H}_p = (\mathbf{Q}\mathbf{U}_p)(\mathbf{U}_p^\top \mathbf{H}). \quad (15)$$

We therefore define

$$\mathbf{L}_p = \mathbf{Q}\mathbf{U}_p \in \mathbb{R}^{d \times p}, \quad \mathbf{R}_p = \mathbf{U}_p^\top \mathbf{H} \in \mathbb{R}^{p \times D}. \quad (16)$$

The full basis  $\mathbf{Q}$  need not be materialized: after zero-padding  $\mathbf{U}_p$  to  $d \times p$ ,  $\mathbf{L}_p = \mathbf{Q}\mathbf{U}_p$  can be obtained by applying the stored Householder reflectors directly. When  $m \leq n$ ,  $\mathbf{M} = \overline{\Delta\mathbf{W}}$ , so these factors already have the required LoRA orientation. When  $m > n$ ,  $\mathbf{M} = \overline{\Delta\mathbf{W}}^\top$ , and the factorization is transposed to recover the original orientation. Thus,

$$(\mathbf{B}_g, \mathbf{A}_g) = \begin{cases} (\mathbf{L}_p, \mathbf{R}_p), & m \leq n, \\ (\mathbf{R}_p^\top, \mathbf{L}_p^\top), & m > n. \end{cases} \quad (17)$$

In both cases,  $\mathbf{B}_g \mathbf{A}_g$  is a best rank- $p$  approximation of the effective weighted aggregate  $\sum_k \alpha_k s_k \mathbf{B}_k \mathbf{A}_k$  by Proposition 1. The resulting pair  $(\mathbf{B}_g, \mathbf{A}_g)$  is therefore a merge-ready factorization as its product already includes the client-side LoRA scaling.

**Complexity.** In the typical regime  $p \leq r \leq d$ , FRAQ requires  $\mathcal{O}((d+D)r^2 + r^3)$  time and  $\mathcal{O}((d+D)r + r^2)$  additional workspace beyond the weighted factor stacks; this bound includes  $\mathcal{O}((d+D)rp)$  for rank- $p$  factor recovery. The orientation in Eq. (12) confines reduced Householder QR to  $\mathbf{X} \in \mathbb{R}^{d \times r}$  and eigendecomposition to  $\mathbf{G} \in \mathbb{R}^{r \times r}$ , while the larger dimension  $D$  appears only in matrix multiplications. We provide derivation and FLOP comparisons in Appendix A.3.

### 3.4 Overall Pipeline

As summarized in Figure 2, FRAQ retains the merge-and-reinitialize workflow of FLoRA (Wang et al., 2024), but broadcasts a recompressed global adapter instead of the full stacked adapter. The rank- $p$  global adapter is stored with  $\alpha_{\text{LoRA},g} = p$ , giving unit global scaling  $s_g = \alpha_{\text{LoRA},g}/p = 1$ . Therefore, merging the broadcast adapter applies exactly the merge-ready patch  $\mathbf{B}_g \mathbf{A}_g$ , without an additional scaling factor. At the beginning of round  $t$ , each client merges the global adapter broadcast in the preceding round into its cached global weights:

$$\mathbf{W}_t = \mathbf{W}_{t-1} + \mathbf{B}_g^{(t-1)} \mathbf{A}_g^{(t-1)}, \quad \mathbf{B}_g^{(0)} \mathbf{A}_g^{(0)} = \mathbf{0}.$$

Each client then initializes a fresh rank- $r_k$  residual adapter  $(\mathbf{B}_k^{(t)}, \mathbf{A}_k^{(t)})$ , fine-tunes it on top of frozen  $\mathbf{W}_t$ , and uploads the resulting factors. Reinitialization prevents the previously merged global adapter from being counted again in the next aggregate. The server forms the exact weighted factor stacks according to Eq. (6) and applies FRAQ to produce

Table 1: Overall comparison of federated LoRA methods on (a) text classification and (b) commonsense reasoning. We report accuracy (%), downlink payload normalized to FLoRA (%), and aggregation latency (ms; mean  $\pm$  std). Best accuracies among compressed-downlink methods are shown in **bold**, and second-best accuracies are underlined. FLoRA and FedEx-LoRA are excluded from this ranking because they do not perform rank recompression.

(a) Text classification.

| METHOD              | Accuracy      |              |              |              |              |              |              |              | DOWNLINK<br>(% FLoRA) | AVG. AGG.<br>TIME (ms) |
|---------------------|---------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-----------------------|------------------------|
|                     | 20 NEWSGROUPS |              | BANKING77    |              | CLINC150     |              | HWU64        |              |                       |                        |
|                     | Dir(0.1)      | Dir(0.02)    | Dir(0.1)     | Dir(0.02)    | Dir(0.1)     | Dir(0.02)    | Dir(0.1)     | Dir(0.02)    |                       |                        |
| FLoRA               | 64.42         | 61.65        | 88.30        | 87.22        | 94.79        | 95.43        | 89.99        | 89.27        | 100.00                | 4.3 ± 0.2              |
| FedEx-LoRA          | 63.02         | 61.62        | 86.96        | 87.00        | 93.38        | 94.35        | 89.78        | 88.85        | 110.00                | 11.7 ± 0.3             |
| FedIT               | <u>65.28</u>  | 56.45        | 84.33        | 79.20        | 91.85        | 89.31        | 88.44        | 82.46        | 10.00                 | 1.9 ± 0.1              |
| LoRA-A <sup>2</sup> | 63.09         | 56.63        | 84.18        | 81.11        | 91.23        | 86.79        | 88.75        | 83.49        | 10.00                 | 1.4 ± 0.1              |
| FlexLoRA            | <b>65.50</b>  | 59.51        | 86.42        | 82.34        | 92.67        | 91.09        | <u>89.58</u> | 83.90        | 10.00                 | 2722.2 ± 7.8           |
| FLoRG               | 61.73         | 54.51        | 81.80        | 77.39        | 88.79        | 84.22        | 87.00        | 79.98        | 1.70                  | 64.4 ± 0.4             |
| FLoRIST             | 62.83         | 56.28        | 83.57        | 78.62        | 88.89        | 84.15        | 88.54        | 81.53        | 4.20                  | 206.1 ± 0.7            |
| FRAQ (τ = 0.80)     | 63.87         | <u>60.95</u> | <u>87.11</u> | <u>85.66</u> | <u>94.42</u> | <u>94.81</u> | <b>89.99</b> | <u>88.65</u> | 23.40                 | 14.2 ± 0.0             |
| FRAQ (τ = 0.95)     | 64.29         | <b>61.16</b> | <b>87.69</b> | <b>86.82</b> | <b>94.57</b> | <b>95.28</b> | <b>89.99</b> | <b>89.06</b> | 48.70                 | 14.3 ± 0.3             |

(b) Reasoning benchmarks.

| METHOD                 | BOOLQ        | PIQA         | SIQA         | HELLASWAG    | WINOGRANDE   | ARC-E        | ARC-C        | OBQA         | AVG.         | DOWNLINK<br>(% FLoRA) | AVG. AGG.<br>TIME (ms) |
|------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-----------------------|------------------------|
| FLoRA                  | 88.78        | 79.49        | 78.45        | 85.34        | 77.27        | 89.73        | 78.24        | 81.60        | 82.36        | 100.00                | 16.7 $\pm$ 0.4         |
| FedEx-LoRA             | 88.75        | 78.89        | 78.15        | 83.42        | 78.30        | 89.69        | 77.22        | 82.60        | 82.13        | 112.51                | 60.0 $\pm$ 9.5         |
| FedIT                  | 87.31        | 78.67        | 76.82        | 79.64        | 73.64        | <b>89.86</b> | <u>77.73</u> | 80.40        | 80.51        | 12.51                 | 9.7 $\pm$ 0.3          |
| LoRA-A <sup>2</sup>    | 85.84        | 71.49        | 72.67        | 55.23        | 64.09        | 81.27        | 63.99        | 74.00        | 71.07        | 12.51                 | 5.0 $\pm$ 0.1          |
| FlexLoRA               | 87.86        | <u>79.49</u> | 76.41        | 81.18        | 75.85        | <b>89.86</b> | <b>78.41</b> | 80.40        | 81.18        | 12.51                 | 197270.4 $\pm$ 34.3    |
| FLoRG                  | 86.97        | 77.42        | 75.13        | 74.50        | 71.19        | 87.37        | 74.32        | 79.60        | 78.31        | 4.01                  | 3403.7 $\pm$ 9.4       |
| FLoRIST                | 85.66        | 77.48        | 73.13        | 70.25        | 71.03        | 88.43        | 75.51        | 76.40        | 77.24        | 12.51                 | 728.0 $\pm$ 6.7        |
| FRAQ ( $\tau = 0.80$ ) | <b>88.47</b> | <b>80.03</b> | <u>77.02</u> | 82.19        | 76.24        | 89.18        | 76.45        | 82.00        | <u>81.45</u> | 40.01                 | 86.7 $\pm$ 1.7         |
| FRAQ ( $\tau = 0.95$ ) | <u>88.13</u> | 79.33        | <b>77.33</b> | <b>83.28</b> | <b>78.45</b> | <u>89.56</u> | 77.39        | <b>82.80</b> | <b>82.03</b> | 67.82                 | 86.7 $\pm$ 1.8         |

the next compact global adapter ( $B_g^{(t)}, A_g^{(t)}$ ) for broadcast.

The weighted factor stacks are algebraically equivalent to explicitly aggregating the full-size local updates. Consequently, FRAQ recompresses the same aggregate without requiring a common local factor rank, naturally accommodating heterogeneous client ranks  $\{r_k\}$ . The complete round-by-round procedure is provided in Appendix B.

## 4 Experiments

In this section, we evaluate the performance of our FRAQ against existing federated LoRA methods across various heterogeneity and homogeneous rank settings and datasets. We assess performance based on accuracy, downlink overhead, and server-side recompression efficiency.

### 4.1 Experimental Setup

**Tasks, federated settings, and metrics.** We evaluate FRAQ with RoBERTa-base (Liu et al., 2019b) on four intent and topic classification datasets:

BANKING77 (Casanueva et al., 2020), 20 News-groups (Lang, 1995), CLINC150 (Larson et al., 2019), and HWU64 (Liu et al., 2019a). Each dataset is partitioned across 10 clients using Dirichlet label splits with  $\alpha \in \{0.1, 0.02\}$ , representing moderate and extreme heterogeneity. Training runs for 30 communication rounds with 200 local steps per round. For this model, we use a single linear classification head for all methods. The classifier is jointly trained with the local LoRA adapters and aggregated using FedAvg during the first five communication rounds, after which it is frozen. This ensures that subsequent performance differences primarily reflect the LoRA adaptation and aggregation mechanisms.

We fine-tune LLaMA-3.2-3B (Meta AI, 2024) on the Commonsense-170K reasoning benchmark following FedEx-LoRA (Singhal et al., 2025). This setting uses eight clients with a pathological cross-task partition, where each client holds one reasoning task. The main evaluation uses a uniform local LoRA rank of 8. We additionally con-

Table 2: Heterogeneous-rank results on commonsense reasoning benchmarks. We report accuracy (%), downlink payload normalized to FLoRA (%), and aggregation latency (ms; mean  $\pm$  std). Best accuracies among compressed-downlink methods are shown in **bold**, and second-best accuracies are underlined.

| METHOD                 | BoolQ        | PIQA         | SIQA         | HELLASWAG    | WINOGRANDE   | ARC-E        | ARC-C        | OBQA         | AVG.         | DOWNLINK<br>(% FLoRA) | AVG. AGG.<br>TIME (ms) |
|------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-----------------------|------------------------|
| FLoRA                  | 89.30        | 76.77        | 77.64        | 81.05        | 74.27        | 88.01        | 72.18        | 78.60        | 79.73        | 100.0                 | 18.9 $\pm$ 0.7         |
| FlexLoRA               | 88.62        | 77.04        | 75.69        | 79.99        | 72.69        | <b>88.13</b> | 69.71        | 75.60        | 78.44        | 27.6                  | 334587.6 $\pm$ 1576.2  |
| FLoRIST                | 88.47        | 76.99        | 74.92        | 79.71        | <u>73.01</u> | <u>88.09</u> | <u>71.76</u> | <u>77.60</u> | <u>78.82</u> | 23.0                  | 4364.6 $\pm$ 10.5      |
| FRAQ ( $\tau = 0.80$ ) | 88.62        | 76.22        | 75.08        | 78.16        | 72.14        | 87.67        | <b>71.93</b> | 76.60        | 78.30        | 16.2                  | 369.4 $\pm$ 5.3        |
| FRAQ ( $\tau = 0.95$ ) | <b>88.93</b> | <b>77.31</b> | <b>76.10</b> | <b>81.01</b> | <b>74.19</b> | 87.63        | 71.33        | <b>78.20</b> | <b>79.34</b> | 45.5                  | 367.9 $\pm$ 4.0        |

sider heterogeneous local ranks  $(r_1, \dots, r_8) = (4, 16, 32, 64, 4, 16, 32, 64)$  under the same reasoning setup, with all other training settings unchanged. Dataset details and complete training hyperparameters are provided in Appendices C.1 and C.3.

We evaluate FRAQ at energy thresholds  $\tau = 0.80$  and  $0.95$  to characterize the accuracy–communication trade-off. We measure per-client downlink by the number of LoRA parameters broadcast per round, normalized to the uncompressed FLoRA stack (100%). We evaluate server-side aggregation efficiency using wall-clock latency and peak memory, complemented by the theoretical FLOP analysis in Appendix A.3. Latency is reported as mean  $\pm$  standard deviation and includes recompression where applicable. Federated training is simulated on a cluster of NVIDIA GH200 GPUs, whereas server-side efficiency is measured on a single GPU from this cluster under identical software conditions for all methods. Unless otherwise noted, weight matrices with compatible shapes are processed using batched GPU kernels.

**Baselines.** We compare FRAQ against factor-space aggregation methods FedIT (Zhang et al., 2024) and LoRA- $A^2$  (Koo et al., 2025), exact and uncompressed aggregation methods FLoRA (Wang et al., 2024) and FedEx-LoRA (Singhal et al., 2025), and recompression methods FlexLoRA (Bai et al., 2024), FLoRG (Meng et al., 2026), and FLoRIST (Ramesh and Dass, 2026a). Detailed baseline descriptions are provided in Appendix C.2. The heterogeneous-rank evaluation includes only methods that natively support varying client ranks, namely FLoRA, FlexLoRA, FLoRIST, and FRAQ.

## 4.2 Main Results

**Accuracy and communication.** Table 1 reports accuracy, normalized downlink, and aggregation latency. Boldface and underlining mark the best and second-best compressed results; unrecompressed

FLoRA and FedEx-LoRA are excluded. FRAQ ( $\tau=0.95$ ) achieves the best compressed result on three of four classification datasets under moderate heterogeneity and all four under extreme heterogeneity, remaining within 0.61 percentage points of FLoRA across all eight settings. FRAQ ( $\tau=0.80$ ) ranks second on all four datasets under extreme heterogeneity. On commonsense reasoning, the two variants rank first and second among compressed methods at 82.03% and 81.45%, respectively, with  $\tau=0.95$  only 0.33 points below FLoRA.

At  $\tau=0.80$ , FRAQ uses 23.4% and 40.0% of the FLoRA downlink for classification and reasoning, respectively;  $\tau=0.95$  increases these values to 48.7% and 67.8% in exchange for higher accuracy. FLoRG and FLoRIST use less downlink but incur substantially larger accuracy losses, particularly under extreme label heterogeneity.

**Heterogeneous client ranks.** Table 2 evaluates methods that natively support heterogeneous local ranks. FRAQ ( $\tau=0.95$ ) achieves the highest average accuracy among compressed methods at 79.34%, only 0.39 percentage points below FLoRA while using 45.5% of its downlink. FRAQ ( $\tau=0.80$ ) reduces the downlink further to 16.2%, the lowest among all compared methods, while retaining an average accuracy of 78.30%.

**Server-side efficiency.** FRAQ requires 14.2 ms and 86.7 ms per aggregation for RoBERTa-base and LLaMA-3.2-3B, respectively, making it 8–15 $\times$  faster than FLoRIST and 190–2,300 $\times$  faster than FlexLoRA. In the heterogeneous-rank experiments, both variants require approximately 0.37 s, around 12 $\times$  faster than FLoRIST and more than 900 $\times$  faster than FlexLoRA.

Figure 3 further examines scalability with total stacked rank and hidden size. Across both sweeps, FRAQ remains the fastest method while maintaining low peak memory under batched and

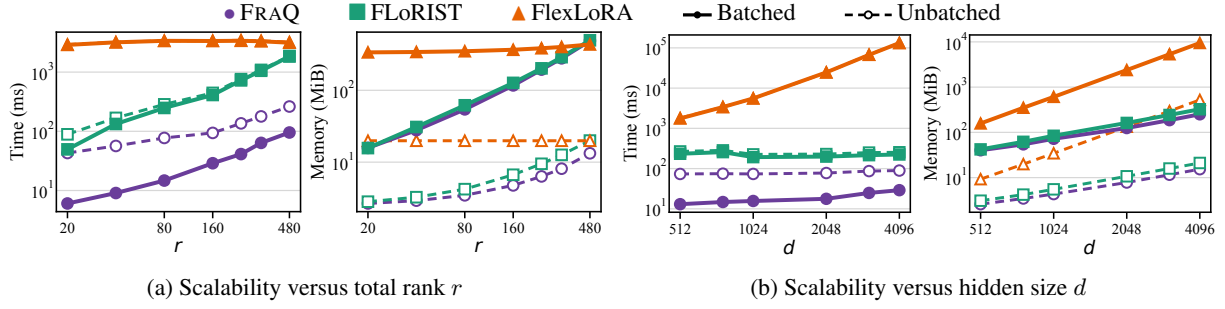


Figure 3: Server-side recompression runtime and peak memory as (a) the total stacked rank  $r$  varies at  $d = 768$  and (b) the hidden size  $d$  varies at  $r = 80$ . Solid and dashed curves denote batched and unbatched execution, respectively. FRAQ consistently achieves the lowest runtime while maintaining low peak memory across both sweeps.

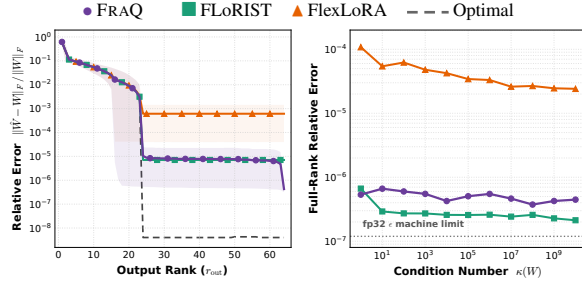


Figure 4: Numerical fidelity and stability in FP32 against FP64 references. The left panel reports mean error over LLaMA-3.2-3B LoRA updates (shading: 10th–90th percentiles), while the right tests full-rank reconstruction under controlled condition numbers. FRAQ tracks the Eckart–Young optimum and remains near  $10^{-7}$  through  $\kappa(W) = 10^{10}$ .

unbatched execution. Unlike FlexLoRA, whose dense reconstruction causes runtime and memory to grow rapidly with hidden size, FRAQ operates on stacked factors; its smaller-side QR and Gram recovery also avoid the repeated rectangular SVDs used by FLoRIST. These trends are consistent with the FLOP analysis in Appendix A.3.

**Numerical fidelity and stability.** Figure 4 evaluates all methods in FP32 with TF32 disabled against FP64 references. The left panel uses 196 LoRA modules from one LLaMA-3.2-3B federated round with eight rank-8 clients. FRAQ closely tracks the Eckart–Young optimum over the signal-bearing rank range and matches the fidelity of FLoRIST. The sharp decrease in optimal error near rank 24, despite a formal stacked rank of 64, indicates substantial spectral redundancy among client updates. The right panel isolates finite-precision stability using rank-64 synthetic aggregates with condition numbers from 1 to  $10^{10}$ . Because the output rank is also 64, the measured error contains no intentional truncation error.

Table 3: Ablation study on real LLaMA-3.2-3B LoRA updates. Relative error is measured against the coordinate-SVD rank-8 reference.

| Variant                   | Time (ms) ↓  | Memory (MiB) ↓ | Rel. error ↓          |
|---------------------------|--------------|----------------|-----------------------|
| Adaptive + Gram           | <b>14.21</b> | <b>90.13</b>   | $2.56 \times 10^{-4}$ |
| Fixed + Gram              | 25.98        | <b>90.13</b>   | $8.51 \times 10^{-4}$ |
| Adaptive + coordinate SVD | 62.26        | 199.02         | Reference             |

FRAQ maintains a relative error of approximately  $10^{-7}$  throughout the sweep, close to FLoRIST and roughly two orders of magnitude below FlexLoRA, demonstrating stable FP32 reconstruction even for severely ill-conditioned aggregates.

**Ablation study.** Table 3 isolates the two main algorithmic choices on 56 LLaMA-3.2-3B gate and up projections of shape  $8192 \times 3072$ . Direction-adaptive orientation reduces latency by  $1.83\times$  without increasing peak memory, while Gram-based recovery is  $4.38\times$  faster and uses  $54.7\%$  less memory than coordinate SVD. Both Gram variants remain within  $0.1\%$  of the rank-8 reference.

## 5 Conclusion

We presented FRAQ, a coordinate-space recompression framework for federated LoRA aggregation. Starting from weighted stacks that exactly represent the aggregated update, FRAQ uses a reduced QR factorization and a compact Gram eigenproblem to recover its spectrum without materializing the dense update or performing a full-matrix SVD. Experiments across diverse federated fine-tuning settings demonstrate that FRAQ achieves a favorable trade-off among model accuracy, downlink communication, and server-side aggregation latency, making federated fine-tuning more practical in heterogeneous and resource-constrained environments.

## Limitations

We note two limitations of FRAQ. First, the cost of coordinate-space aggregation scales with the total stacked rank of the participating clients in each round, that is, the sum of their local LoRA ranks. When a large cohort of clients participates per round, this total rank grows accordingly, and the reduced QR factorization, the Gram-matrix construction, and the eigendecomposition all become more expensive. Consequently, the server-side recompression efficiency degrades as the number of participating clients increases, and the advantage of FRAQ over full-weight SVD shrinks in the regime of very large per-round participation. A detailed cost breakdown is given in the appendix.

Second, like FLoRA, FRAQ follows a merge-and-reinitialize scheme: in every round the consolidated update is baked into the global model and clients reinitialize fresh adapters, so local adapters are not warm-started from the previous round as in PEFT-style methods that reuse the downloaded adapter. This can slow per-adapter convergence relative to methods that keep refining a persistent adapter. Importantly, however, the coordinate-space recompression at the heart of FRAQ is not tied to the merge-and-reinitialize paradigm. The same QR-and-Gram construction could potentially replace SVD-based recompression in non-merge-and-reinitialize methods such as FLoRIST and FLoRG, but validating the resulting accuracy and runtime is left to future work.

## References

- Jiamu Bai, Daoyuan Chen, Bingchen Qian, Liuyi Yao, and Yaliang Li. 2024. [Federated fine-tuning of large language models under heterogeneous tasks and client resources](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Tom Brown et al. 2020. [Language models are few-shot learners](#). In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- Inigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. 2020. Efficient intent detection with dual sentence encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI*, pages 38–45. Association for Computational Linguistics.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.
- Gene H Golub, Alan Hoffman, and Gilbert W Stewart. 1987. A generalization of the eckart-young-mirsky matrix approximation theorem. *Linear Algebra and its applications*, 88:317–327.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, et al. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Jeremy Howard and Sebastian Ruder. 2018. [Universal language model fine-tuning for text classification](#). In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 328–339, Melbourne, Australia. Association for Computational Linguistics.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](#). In *International Conference on Learning Representations*.
- Zhiqiang Hu, Lei Wang, Yihuai Lan, Wanyu Xu, Eepeng Lim, Lidong Bing, Xing Xu, Soujanya Poria, and Roy Ka-Wei Lee. 2023. LLM-Adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Jabin Koo, Minwoo Jang, and Jungseul Ok. 2025. [Towards robust and efficient federated low-rank adaptation with heterogeneous clients](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 416–429, Vienna, Austria. Association for Computational Linguistics.
- Ken Lang. 1995. Newsweeder: Learning to filter netnews. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 331–339.
- Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, Andrew Lee, Parker Hill,

- Jonathan K. Kummerfeld, Kevin Leach, Michael A. Laurenzano, Lingjia Tang, and Jason Mars. 2019. An evaluation dataset for intent classification and out-of-scope prediction. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 1311–1316. Association for Computational Linguistics.
- Xingkun Liu, Arash Eshghi, Pawel Swietojanski, and Verena Rieser. 2019a. Benchmarking natural language understanding services for building conversational agents. In *Proceedings of the Tenth International Workshop on Spoken Dialogue Systems Technology*, Ortigia, Siracusa, Italy. Springer.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019b. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. [Communication-Efficient Learning of Deep Networks from Decentralized Data](#). In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR.
- Chuiyang Meng, Ming Tang, and Vincent W.S. Wong. 2026. [FLoRG: Federated fine-tuning with low-rank gram matrices and procrustes alignment](#). In *The Fourteenth International Conference on Learning Representations*.
- Meta AI. 2024. [Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models](#).
- Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? A new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Aniello Panariello, Daniel Marczak, Simone Magistri, Angelo Porrello, Bartłomiej Twardowski, Andrew D. Bagdanov, Simone Calderara, and Joost van de Weijer. 2025. [Accurate and efficient low-rank merging in core space](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Qwen Team. 2025. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.
- Hariharan Ramesh and Jyotikrishna Dass. 2026a. [FLoRIST: Singular value thresholding for efficient and accurate federated fine-tuning of large language models](#). In *Ninth Conference on Machine Learning and Systems*.
- Hariharan Ramesh and Jyotikrishna Dass. 2026b. [Spectral transformation for layer-wise global rank discovery in federated lora for vision transformers](#). *Preprint*, arXiv:2607.21074.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. WinoGrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.
- Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Social IQa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Raghav Singhal, Kaustubh Ponshe, and Praneeth Vepakomma. 2025. [FedEx-LoRA: Exact aggregation for federated and efficient fine-tuning of large language models](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1316–1336, Vienna, Austria. Association for Computational Linguistics.
- Youbang Sun, Zitao Li, Yaliang Li, and Bolin Ding. 2024. [Improving lora in privacy-preserving federated learning](#). In *The Twelfth International Conference on Learning Representations*.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. [Llama: Open and efficient foundation language models](#). *arXiv preprint arXiv:2302.13971*.
- Ziyao Wang, Zheyu Shen, Yexiao He, Guoheng Sun, Hongyi Wang, Lingjuan Lyu, and Ang Li. 2024. Flora: Federated fine-tuning large language models with heterogeneous low-rank adaptations. In *Advances in Neural Information Processing Systems*, volume 37, pages 22513–22533.
- Peishen Yan, Yang Hua, Hao Wang, Jiaru Zhang, Xiaoyu Wu, Tao Song, and Haibing Guan. 2026. [Fed-momentum: Preserving lora training momentum in federated fine-tuning](#). *Preprint*, arXiv:2603.08014.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Jianyi Zhang, Saeed Vahidian, Martin Kuo, Chunyuan Li, Ruiyi Zhang, Tong Yu, Guoyin Wang, and Yiran Chen. 2024. [Towards building the federatedgpt: Federated instruction tuning](#). In *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6915–6919.

## A Theoretical Guarantees and Complexity Analysis

### A.1 Proof of Proposition 1

*Proof.* Because  $\text{col}(\mathbf{X}) \subseteq \text{col}(\mathbf{Q})$ , the matrix  $\mathbf{Q}\mathbf{Q}^\top$  acts as the identity on  $\mathbf{X}$ :

$$\mathbf{Q}\mathbf{Q}^\top \mathbf{X} = \mathbf{X}. \quad (18)$$

Therefore,

$$\mathbf{Q}\mathbf{H} = \mathbf{Q}\mathbf{Q}^\top \mathbf{X}\mathbf{Y} = \mathbf{X}\mathbf{Y} = \mathbf{M}. \quad (19)$$

Moreover, for any  $\widehat{\mathbf{H}} \in \mathbb{R}^{r \times D}$ ,

$$\begin{aligned} \|\mathbf{M} - \mathbf{Q}\widehat{\mathbf{H}}\|_F &= \|\mathbf{Q}(\mathbf{H} - \widehat{\mathbf{H}})\|_F \\ &= \|\mathbf{H} - \widehat{\mathbf{H}}\|_F, \end{aligned}$$

where the last equality follows from  $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}_r$ . This proves Eq. (10).

Let  $\mathbf{H} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$  be a compact SVD. Since both  $\mathbf{Q}$  and  $\mathbf{U}$  have orthonormal columns,

$$(\mathbf{Q}\mathbf{U})^\top (\mathbf{Q}\mathbf{U}) = \mathbf{U}^\top \mathbf{Q}^\top \mathbf{Q}\mathbf{U} = \mathbf{I}. \quad (20)$$

Therefore  $\mathbf{M} = (\mathbf{Q}\mathbf{U})\mathbf{\Sigma}\mathbf{V}^\top$  is a compact SVD of  $\mathbf{M}$ . Thus,  $\mathbf{H}$  and  $\mathbf{M}$  share the same nonzero singular values.

Truncating this SVD to its leading  $p$  components and applying the Eckart–Young–Mirsky theorem (Golub et al., 1987) proves Eq. (11).  $\square$

### A.2 General-Rank Formulation and Asymptotic Complexity

For completeness, we first analyze the general case in which the total stacked rank may exceed the smaller weight dimension. Let

$$\begin{aligned} d &= \min(m, n), \quad D = \max(m, n), \\ r &= \sum_{k=1}^K r_k, \quad s = \min(d, r). \end{aligned} \quad (21)$$

After orientation,  $\mathbf{X} \in \mathbb{R}^{d \times r}$  and  $\mathbf{Y} \in \mathbb{R}^{r \times D}$ . A reduced Householder QR factorization obtains  $\mathbf{R} \in \mathbb{R}^{s \times r}$  and an implicit representation of  $\mathbf{Q} \in \mathbb{R}^{d \times s}$ , followed by  $\mathbf{H} = \mathbf{R}\mathbf{Y} \in \mathbb{R}^{s \times D}$  and  $\mathbf{G} = \mathbf{H}\mathbf{H}^\top \in \mathbb{R}^{s \times s}$ . The per-matrix costs are

$$\text{reduced QR: } \mathcal{O}(drs), \quad (22)$$

$$\text{coordinate construction: } \mathcal{O}(srD), \quad (23)$$

$$\text{Gram construction: } \mathcal{O}(s^2D), \quad (24)$$

$$\text{eigendecomposition: } \mathcal{O}(s^3), \quad (25)$$

$$\text{patch recovery: } \mathcal{O}(ps(d+D)). \quad (26)$$

Thus, the formulation remains valid when  $r > d$ , in which case  $s = d$ . In the typical regime  $r \leq d$ , we have  $s = r$ , and the total time reduces to  $\mathcal{O}(dr^2 + r^2D + r^3)$ , typically dominated by the coordinate and Gram constructions.

The direction-adaptive orientation ensures that the Householder representation of  $\mathbf{Q}$  is computed only from the factor associated with the smaller dimension  $d$ . If the factorization were always anchored on  $\mathbf{B}$ , QR would instead cost  $\mathcal{O}(Dr^2)$  when  $m > n$ , compared with FRAQ’s  $\mathcal{O}(dr^2)$ , a factor-of- $D/d$  reduction. The additional workspace for the Householder representation,  $\mathbf{R}$ ,  $\mathbf{H}$ , and  $\mathbf{G}$  is  $\mathcal{O}(s(d+D+r) + s^2)$ , which reduces to  $\mathcal{O}((d+D)r + r^2)$  in the typical regime. These bounds exclude the resident stacked factors representing the exact aggregate. Direct dense recompression additionally materializes the  $d \times D$  update, costing  $\mathcal{O}(dDr)$  computation and  $\mathcal{O}(dD)$  workspace before its matrix decomposition.

For  $L$  adapted weight matrices, the total server cost is the sum of the per-matrix costs,

$$F_{\text{total}} = \sum_{\ell=1}^L F(d_\ell, D_\ell, r_\ell, p_\ell), \quad (27)$$

which accounts for the different shapes and selected ranks of transformer projections.

### A.3 Detailed FLOP Comparison

We next provide a leading-order arithmetic comparison among FRAQ, FLoRIST, and FlexLoRA. Let  $p \leq r$  denote the retained global rank. We focus on the typical LoRA regime

$$p \leq r \ll d \leq D. \quad (28)$$

For a coarse leading-order comparison, we use the following standard arithmetic estimates:

$$F_{\text{SVD}}(a, r) \approx 4ar^2 + \mathcal{O}(r^3), \quad (29)$$

$$F_{\text{QR}}(a, r) \approx 2ar^2 - \frac{2}{3}r^3, \quad (30)$$

$$F_{\text{GEMM}}(a, b, c) \approx 2abc. \quad (31)$$

Here, Eq. (29) approximates a thin SVD of an  $a \times r$  matrix, Eq. (30) approximates reduced Householder QR, and Eq. (31) approximates multiplication of an  $a \times b$  matrix by a  $b \times c$  matrix. These estimates are intended only to expose the leading dimension-dependent terms; exact constants depend on the numerical routine, whether singular vectors are explicitly formed, and the underlying hardware implementation.

**FlexLoRA.** FlexLoRA explicitly materializes the dense aggregate  $\overline{\Delta\mathbf{W}} \in \mathbb{R}^{d \times D}$  before decomposition. Even under a favorable implementation that forms the aggregate through one multiplication of the weighted stacked factors, dense materialization costs approximately  $2dDr$  FLOPs. An economy SVD of the resulting  $d \times D$  matrix, with  $d \leq D$ , has a leading cost of approximately  $4Dd^2 + \mathcal{O}(d^3)$ . Recovering the top- $p$  factors requires only lower-order scaling and slicing operations. We therefore obtain

$$F_{\text{FlexLoRA}} \approx 2dDr + 4Dd^2 + \mathcal{O}(d^3 + (d + D)p). \quad (32)$$

This estimate is conservative in favor of FlexLoRA because it omits any additional accumulation cost incurred when client updates are materialized separately.

**FLoRIST.** FLoRIST first applies thin SVDs to the two stacked factors, whose dimensions are  $d \times r$  and  $D \times r$  after orienting them consistently. Their leading cost is

$$4dr^2 + 4Dr^2 = 4(d + D)r^2. \quad (33)$$

FLoRIST then forms and decomposes an  $r \times r$  intermediate matrix, contributing an additional  $\mathcal{O}(r^3)$  term. Recovering rank- $p$  global factors requires multiplying the rectangular singular-vector bases by the retained intermediate factors, contributing approximately  $2(d + D)rp$  FLOPs. Its total leading cost is therefore

$$F_{\text{FLoRIST}} \approx 4(d + D)r^2 + 2(d + D)rp + \mathcal{O}(r^3). \quad (34)$$

**FRAQ.** FRAQ performs reduced QR only on the factor anchored along the smaller weight dimension. The QR factorization of the  $d \times r$  matrix costs approximately  $2dr^2 - \frac{2}{3}r^3$  FLOPs. Constructing the coordinate matrix

$$\mathbf{H} = \mathbf{R}\mathbf{Y}, \quad \mathbf{R} \in \mathbb{R}^{r \times r}, \quad \mathbf{Y} \in \mathbb{R}^{r \times D}, \quad (35)$$

costs approximately  $2Dr^2$  FLOPs. Forming the symmetric Gram matrix  $\mathbf{G} = \mathbf{H}\mathbf{H}^\top$  with a symmetry-aware rank- $k$  update costs approximately  $Dr^2$  FLOPs; a generic matrix multiplication that explicitly computes both triangular halves would instead cost approximately  $2Dr^2$ .

The symmetric eigendecomposition of  $\mathbf{G} \in \mathbb{R}^{r \times r}$  contributes  $\mathcal{O}(r^3)$ . Finally, recovering the rank- $p$  merge factors requires applying the

stored Householder reflectors to  $\mathbf{U}_p$  and computing  $\mathbf{U}_p^\top \mathbf{H}$ , with a combined cost of  $\mathcal{O}((d + D)rp)$ . Thus, when the Gram matrix is formed using symmetry, FRAQ has the leading cost

$$F_{\text{FRAQ}} \approx (2d + 3D)r^2 + \mathcal{O}((d + D)rp + r^3). \quad (36)$$

If the full Gram matrix is formed with generic GEMM, the coefficient  $3D$  in Eq. (36) becomes  $4D$ .

**Comparison with FLoRIST.** Equations (34) and (36) show that FRAQ and FLoRIST share the same coarse asymptotic order,  $\Theta(Dr^2 + r^3)$ , but FRAQ performs substantially less decomposition-heavy computation. FLoRIST applies thin SVDs to both rectangular factors, whereas FRAQ applies reduced QR only to the factor associated with the smaller weight dimension. The rectangular decomposition FLOPs are

$$F_{\text{dec}}^{\text{FLoRIST}} \approx 4(d + D)r^2, \quad (37)$$

$$F_{\text{dec}}^{\text{FRAQ}} \approx 2dr^2. \quad (38)$$

Consequently, FRAQ reduces the dimension-dependent decomposition workload by approximately

$$\frac{F_{\text{dec}}^{\text{FLoRIST}}}{F_{\text{dec}}^{\text{FRAQ}}} \approx 2 \left( 1 + \frac{D}{d} \right). \quad (39)$$

For square matrices, Eq. (39) corresponds to an approximately  $4\times$  reduction in rectangular decomposition FLOPs. The reduction increases for highly rectangular projection matrices.

Considering all leading  $r^2$  terms and neglecting the common rank- $p$  reconstruction term, the arithmetic ratio is approximately

$$\frac{F_{\text{FLoRIST}}}{F_{\text{FRAQ}}} \approx \frac{4(d + D)}{2d + 3D}. \quad (40)$$

For  $D \geq d$ , this ratio ranges from approximately  $1.33\times$  to  $1.60\times$ . The larger empirical wall-clock gains arise because FRAQ handles the dependence on  $D$  through GEMM/SYRK kernels rather than additional SVDs, which generally have higher synchronization, workspace, and kernel-launch overhead.

**Comparison with FlexLoRA.** The advantage over FlexLoRA is also asymptotic. FlexLoRA's dense SVD contributes the dominant term  $\Theta(Dd^2)$ , whereas FRAQ operates on the stacked rank and requires  $\Theta(Dr^2)$  work in the typical regime. Because

Table 4: Leading-order server-side FLOP comparison for one adapted weight matrix. We assume  $p \leq r \ll d \leq D$ , where  $d = \min(m, n)$ ,  $D = \max(m, n)$ , and  $r = \sum_k r_k$ . The FRAQ Gram matrix is assumed to be formed with a symmetry-aware rank- $k$  update. The displayed FLOPs are approximate leading-order quantities rather than exact operation counts.

| Method   | Matrix factorizations                                           | Leading-order FLOPs                           | Dominant scaling     |
|----------|-----------------------------------------------------------------|-----------------------------------------------|----------------------|
| FlexLoRA | SVD( $d \times D$ )                                             | $2dDr + 4Dd^2 + \mathcal{O}(d^3 + (d + D)p)$  | $\Theta(Dd^2 + dDr)$ |
| FLoRIST  | SVD( $d \times r$ ) + SVD( $D \times r$ ) + SVD( $r \times r$ ) | $4(d + D)r^2 + 2(d + D)rp + \mathcal{O}(r^3)$ | $\Theta(Dr^2 + r^3)$ |
| FRAQ     | QR( $d \times r$ ) + EVD( $r \times r$ )                        | $(2d + 3D)r^2 + \mathcal{O}((d + D)rp + r^3)$ | $\Theta(Dr^2 + r^3)$ |

$r \ll d$ , FRAQ replaces quadratic dependence on the smaller model dimension  $d$  with quadratic dependence on the stacked rank  $r$ . Let  $\gamma = D/d \geq 1$ . Neglecting the lower-order dense-materialization, rank- $p$  reconstruction, and cubic terms, the approximate FLOP ratio is

$$\frac{F_{\text{FlexLoRA}}}{F_{\text{FRAQ}}} \approx \frac{4\gamma}{2 + 3\gamma} \left(\frac{d}{r}\right)^2. \quad (41)$$

Thus, the arithmetic advantage of FRAQ over FlexLoRA grows quadratically with the dimension-to-rank ratio  $d/r$ , consistent with the scalability results in Figure 3.

## B Complete FRAQ Procedure

Algorithm 1 provides the complete round-by-round procedure for FRAQ, including client-side merge-and-reinitialization and server-side recompression. Reinitializing the residual adapters after merging the previous global adapter prevents their old factorization from being counted again against the updated global base. Although our storage convention gives unit global scaling, Algorithm 1 writes  $s_g$  explicitly in each merge operation.

### Algorithm 1: End-to-end federated training with FRAQ.

**Input:** Pretrained weights  $\mathbf{W}_0$ , rounds  $T$ , clients  $\mathcal{C}$  with ranks  $\{r_k\}$ , LoRA scales  $\{s_k\}$ , dataset sizes  $\{n_k\}$ , energy threshold  $\tau \in (0, 1]$   
**Output:** Merged global model  $\mathbf{W}_{T+1}$   
 $\mathbf{W}_1 \leftarrow \mathbf{W}_0$   
 $s_g^{(0)} \mathbf{B}_g^{(0)} \mathbf{A}_g^{(0)} \leftarrow \mathbf{0}$   
**for**  $t = 1$  **to**  $T$  **do**  
    **Server:** broadcast the round- $(t-1)$  global adapter  $(\mathbf{B}_g^{(t-1)}, \mathbf{A}_g^{(t-1)}, s_g^{(t-1)})$  to all clients (empty when  $t = 1$ )  
    **foreach**  $k \in \mathcal{C}$  **do in parallel**  
        Merge the received global adapter:  
         $\mathbf{W}_t \leftarrow \mathbf{W}_{t-1} + s_g^{(t-1)} \mathbf{B}_g^{(t-1)} \mathbf{A}_g^{(t-1)}$   
        Initialize a fresh residual adapter  $(\mathbf{B}_k, \mathbf{A}_k)$  of rank  $r_k$   
         $(\mathbf{B}_k, \mathbf{A}_k) \leftarrow \text{LocalUpdate}(\mathbf{W}_t, \mathbf{B}_k, \mathbf{A}_k)$   
        Upload  $(\mathbf{B}_k, \mathbf{A}_k)$  to server  
    **Server:** form scaled and weighted stacks  
     $\mathbf{B} \leftarrow [\sqrt{\alpha_1} \mathbf{B}_1, \dots, \sqrt{\alpha_K} \mathbf{B}_K]$   
     $\mathbf{A} \leftarrow [\sqrt{\alpha_1} s_1 \mathbf{A}_1; \dots; \sqrt{\alpha_K} s_K \mathbf{A}_K]$   
    Orient:  $(\mathbf{X}, \mathbf{Y}) \leftarrow (\mathbf{B}, \mathbf{A})$  if  $m \leq n$ ; else  $(\mathbf{X}, \mathbf{Y}) \leftarrow (\mathbf{A}^\top, \mathbf{B}^\top)$   
     $s \leftarrow \min(d, r)$   
    Compute Householder QR of  $\mathbf{X}$ , storing reflectors and  $\mathbf{R}$   
    Form  $\mathbf{H} \leftarrow \mathbf{R}\mathbf{Y}$  and  $\mathbf{G} \leftarrow \mathbf{H}\mathbf{H}^\top$   
    Eigendecompose  $\mathbf{G} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$  and order the eigenpairs by decreasing  $\lambda_i$   
    Select the smallest  $p \in \{1, \dots, s\}$  such that  $\sum_{i=1}^p \lambda_i / \sum_{i=1}^s \lambda_i \geq \tau$ ; retain  $\mathbf{U}_p$   
     $\mathbf{L}_p \leftarrow \text{ApplyQ}(\text{reflectors}, \mathbf{U}_p)$  and  $\mathbf{R}_p \leftarrow \mathbf{U}_p^\top \mathbf{H}$   
     $(\mathbf{B}_g^{(t)}, \mathbf{A}_g^{(t)}) \leftarrow (\mathbf{L}_p, \mathbf{R}_p)$  if  $m \leq n$ ; else  $(\mathbf{B}_g^{(t)}, \mathbf{A}_g^{(t)}) \leftarrow (\mathbf{R}_p^\top, \mathbf{L}_p^\top)$   
    Set  $\alpha_{\text{LoRA},g}^{(t)} \leftarrow p$  and  $s_g^{(t)} \leftarrow \alpha_{\text{LoRA},g}^{(t)} / p = 1$ ;  
    store the rank- $p$  global adapter  
     $\mathbf{W}_{t+1} \leftarrow \mathbf{W}_t + s_g^{(t)} \mathbf{B}_g^{(t)} \mathbf{A}_g^{(t)}$   
**return**  $\mathbf{W}_{T+1}$

## C Experimental Details

### C.1 Datasets and Data Partitions

This appendix expands on the datasets used in our two evaluation settings: the federated intent/topic classification benchmark (Table 1a) and the com-

monsense reasoning benchmark used in the additional evaluation (Table 1b).

**Intent and topic classification.** For the federated RoBERTa-base experiments we use four English text classification datasets:

- **BANKING77** (Casanueva et al., 2020): a fine-grained single-domain intent detection dataset of online-banking customer queries spanning 77 intents.
- **CLINC150** (Larson et al., 2019): a multi-domain intent classification benchmark covering 150 intents across 10 domains, designed to also include out-of-scope queries.
- **HWU64** (Liu et al., 2019a): a multi-domain home-assistant intent dataset with 64 intents across 21 domains.
- **20 Newsgroups** (Lang, 1995): a classic topic classification benchmark of newsgroup posts grouped into 20 categories.

To emulate realistic client data heterogeneity, each dataset is partitioned across clients with a Dirichlet distribution  $\text{Dir}(\alpha)$  over the label space, where a smaller concentration parameter  $\alpha$  yields a more skewed (non-IID) partition. We report results under a moderate heterogeneity level  $\text{Dir}(0.1)$  and an extreme level  $\text{Dir}(0.02)$ , and evaluate global test accuracy at the final communication round.

**Commonsense reasoning.** For the additional evaluation in Table 1b, we adopt the Commonsense-170K training and evaluation protocol used by FedEx-LoRA (Singhal et al., 2025), following Hu et al. (2023): models are fine-tuned on the Commonsense-170K instruction-tuning corpus and evaluated on eight multiple-choice reasoning tasks. Each task is cast as a multiple-choice problem in which the model selects the correct answer from the provided options, and we report per-task accuracy together with the eight-task average. The benchmarks are:

- **BoolQ** (Clark et al., 2019): naturally occurring yes/no questions, each paired with a Wikipedia passage that provides the context.
- **PIQA** (Bisk et al., 2020): physical commonsense questions that require choosing the more sensible of two candidate solutions to an everyday physical goal.

- **SIQA** (Social IQa) (Sap et al., 2019): questions probing reasoning about people’s social interactions and their likely motivations and consequences.
- **HellaSwag** (Zellers et al., 2019): sentence-completion questions in which the model selects the most plausible continuation of a given context.
- **WinoGrande** (Sakaguchi et al., 2021): a large-scale adversarial Winograd schema benchmark requiring binary pronoun/coreference resolution.
- **ARC-Easy** (ARC-e) and **ARC-Challenge** (ARC-c) (Clark et al., 2018): grade-school science questions split into an easier subset and a harder subset of questions that simple retrieval methods answer incorrectly.
- **OpenBookQA** (OBQA) (Mihaylov et al., 2018): elementary science questions that require combining a small “open book” of core science facts with broad commonsense knowledge.

Because our reasoning evaluation follows the same benchmark protocol as FedEx-LoRA, the results in Table 1b are directly comparable to theirs.

## C.2 Baseline Methods

We compare against all methods reported in Table 1. **FedIT** (Zhang et al., 2024) applies FedAvg independently to the two LoRA factors, while **LoRA-A<sup>2</sup>** (Koo et al., 2025) alternates which LoRA factor is frozen and adaptively selects the rank to improve robustness under heterogeneous data. **FLoRA** (Wang et al., 2024) represents the exact weighted update by stacking the client factors and broadcasts the resulting high-rank adapter. **FedEx-LoRA** (Singhal et al., 2025) provides an alternative exact aggregation method for federated LoRA. **FlexLoRA** (Bai et al., 2024) materializes the dense aggregate and applies truncated SVD to construct compact global adapters. **FLoRG** (Meng et al., 2026) aggregates low-rank Gram matrices and uses Procrustes alignment, whereas **FLoRIST** (Ramesh and Dass, 2026a) recompresses the stacked factors through rectangular SVDs and a small intermediate decomposition.

Table 5: Hyperparameter details for the RoBERTa classification and LLaMA reasoning experiments. Table 1 uses a uniform local LoRA rank of 8; the heterogeneous-rank configuration in Table 2 is specified in a separate row.

| Hyperparameter                                 | RoBERTa-base             | LLaMA-3.2-3B                     |
|------------------------------------------------|--------------------------|----------------------------------|
|                                                | BANKING77, 20Newsgroups, |                                  |
| Training datasets                              | CLINC150, HWU64          | COMMONSENSE170K                  |
| Optimizer                                      | AdamW                    | AdamW                            |
| Learning rate                                  | $5 \times 10^{-4}$       | $3 \times 10^{-4}$               |
| Learning-rate schedule                         | Constant                 | Constant                         |
| Warmup ratio                                   | 0                        | 0                                |
| Batch size (per client)                        | 32                       | 16                               |
| Gradient accumulation steps                    | 1                        | 1                                |
| Local optimizer steps per round                | 200                      | 100                              |
| Global communication rounds                    | 30                       | 5                                |
| Total clients                                  | 10                       | 8                                |
| Client participation rate                      | 1.0                      | 1.0                              |
| Data heterogeneity                             | Dirichlet                | Pathological non-IID             |
| Dirichlet concentration $\alpha$               | 0.1, 0.02                | N/A                              |
| LoRA rank (Table 1)                            | 8                        | 8                                |
| LoRA ranks (Table 2)                           | N/A                      | [4, 16, 32, 64, 4, 16, 32, 64]   |
| LoRA scale $s_k = \alpha_{\text{LoRA},k}/r_k$  | 2                        | 2                                |
| LoRA alpha $\alpha_{\text{LoRA},k}$ (Table 1)  | 16                       | 16                               |
| LoRA alphas $\alpha_{\text{LoRA},k}$ (Table 2) | N/A                      | [8, 32, 64, 128, 8, 32, 64, 128] |
| LoRA dropout                                   | 0                        | 0                                |
| Target modules                                 | query, key,              | q_proj, k_proj, v_proj, o_proj,  |
|                                                | value                    | gate_proj, up_proj, down_proj    |
| Maximum sequence length                        | 256                      | 256                              |
| Training precision                             | FP32                     | BF16                             |
| Random seed                                    | 42                       | 42                               |
| Energy-retention threshold $\tau$              | 0.80, 0.95               | 0.80, 0.95                       |
| Evaluation interval                            | Every 5 rounds           | Post-training evaluation         |
| Classifier-head freezing                       | After round 5            | N/A                              |

### C.3 Hyperparameter Settings

Table 5 lists the training and federated learning hyperparameters for the RoBERTa-base classification experiments and the LLaMA-3.2-3B reasoning experiments. Unless otherwise noted, we use the same setting for all methods and datasets to ensure a controlled comparison.

### D Alternative Orthogonalization Backends

Proposition 1 establishes exact coordinate-space equivalence whenever  $Q$  is an orthonormal basis whose span contains the column space of the oriented left factor  $X$ . However, this proposition does not prescribe how such a basis must be constructed. Although FRAQ uses reduced QR as its default implementation, other exact orthogonalization methods can satisfy the same conditions. Specifically, for any  $Q$  satisfying

$$Q^\top Q = I, \quad \text{col}(X) \subseteq \text{col}(Q),$$

we may define

$$C = Q^\top X, \quad H = CY.$$

Then  $X = QC$  and  $XY = QH$ . By Proposition 1, the compact eigendecomposition

$$HH^\top = U\Lambda U^\top$$

recovers the nonzero singular spectrum and left singular subspace needed by FRAQ. Reduced QR is one realization of this construction: if  $X = Q_{\text{qr}}R$ , then  $C = R$ . The triangular structure of  $R$  is incidental; orthonormality and column-space coverage are the essential conditions.

**Polar factorization.** For full-column-rank  $X$ , the polar decomposition gives

$$\begin{aligned} X &= Q_{\text{pol}}S, \\ Q_{\text{pol}} &= X(X^\top X)^{-1/2}, \\ S &= (X^\top X)^{1/2}. \end{aligned}$$

Here  $Q_{\text{pol}}$  has orthonormal columns and  $S$  is symmetric positive definite. Thus  $XY = Q_{\text{pol}}(SY)$ , and  $S$  can replace the QR factor  $R$  without changing the subsequent coordinate-space decomposition. Hence exact FRAQ depends on an orthogonal-basis factorization, rather than on QR specifically.

**Iterative orthogonalization.** An iterative polar method, such as Newton–Schulz orthogonalization, can instead produce an approximately orthogonal basis  $\tilde{Q}$ . The direct drop-in construction is

$$\begin{aligned}\tilde{C} &= \tilde{Q}^\top X, & \tilde{H} &= \tilde{C}Y, \\ \tilde{H}\tilde{H}^\top &= \tilde{U}\tilde{\Lambda}\tilde{U}^\top.\end{aligned}$$

No triangular structure is required of  $\tilde{C}$ . With a finite number of iterations, however,  $\tilde{Q}^\top \tilde{Q}$  generally differs from  $I$ , and therefore

$$\tilde{Q}\tilde{C} = \tilde{Q}\tilde{Q}^\top X \neq X.$$

This backend consequently introduces an orthogonalization error in addition to the intended rank-truncation error.

For more accurate reconstruction, one may instead use the least-squares coordinates

$$\tilde{C} = (\tilde{Q}^\top \tilde{Q})^{-1} \tilde{Q}^\top X.$$

Then  $\tilde{Q}\tilde{C} = P_{\tilde{Q}}X$ , which equals  $X$  whenever  $\tilde{Q}$  spans the same column space as  $X$ . Importantly, exact reconstruction alone does not make the usual eigendecomposition of  $\tilde{H}\tilde{H}^\top$  exact: when  $\tilde{Q}$  is not orthonormal, left multiplication by  $\tilde{Q}$  changes the metric. An exact small coordinate-space variant can restore the standard form by factoring

$$\tilde{Q}^\top \tilde{Q} = LL^\top, \quad XY = (\tilde{Q}L^{-\top})(L^\top \tilde{C}Y),$$

where  $\tilde{Q}L^{-\top}$  has orthonormal columns. The FRAQ coordinate-space decomposition can then be applied to  $L^\top \tilde{C}Y$ . This only requires a small  $r \times r$  factorization, but it reduces the advantage of a pure GEMM-based backend.

**Accuracy and performance considerations.** An iterative method run sufficiently close to convergence can replace thin QR nearly transparently. In contrast, a fixed-step, low-precision scheme designed primarily for optimizer updates need not deliver the accuracy required for recompression. We therefore recommend measuring

$$\|\tilde{Q}^\top \tilde{Q} - I\|_F, \quad \frac{\|X - \tilde{Q}(\tilde{Q}^\top X)\|_F}{\|X\|_F},$$

as well as the gap between the final truncated update and exact QR-based FRAQ. Whether several Newton–Schulz GEMMs outperform a vendor-optimized thin QR depends on the tall-skinny dimensions, precision, and GPU. The algorithmic substitution is therefore valid, while its practical benefit must be established by an accuracy–latency benchmark on the target hardware.